

User Stories

Stakeholder Map

Stakeholder	Type
Software Engineer (cloud team)	Primary
Consumer using SmartHQ connected ecosystem	Secondary
GE Appliances call center operator	Hidden

User Stories

- US-01 (primary): As a Software Engineer on the cloud team, I want to increase my output so that I can contribute to key projects faster.
 - Independent
 - Is not blocked by any other initiative or project. The team has bandwidth to start something new.
 - Negotiable
 - Methods to increase developer efficiency are not defined, but are ready to be explored.
 - Valuable
 - Allows stakeholder to grow professionally, increase reputation at the organization, and gain a wider variety of experience working on different features.
 - Estimable
 - Industry-wide precedent exists for boosting developer output, allowing the team to approximate the effort needed.
 - Small
 - Efficient design allows this user story to fit into the scope of a single sprint.
 - Testable
- US-02 (secondary): As a consumer of GE Appliances' SmartHQ connected ecosystem, I want to use the smart features of my appliance without interruption so I can get my money's worth.
 - Independent
 - Work does not rely on any existing or planned user-facing feature.
 - Negotiable
 - Options open between increasing reliability, switching between primary/secondary devices, new methods to stay connected, etc.
 - Valuable
 - User satisfaction and usability are priorities for financial growth and technical reliability.
 - Estimable
 - Expert product knowledge from industry insiders will narrow the scope of this user story into work measurable by a project manager and the team during scrum poker.
 - Small
 - Input from industry insiders above will narrow the scope of user needs enough to fit inside one working sprint.

- US-03 (hidden): As a GE Appliances call center operator, I want to limit the amount of cloud failures in production so call volumes stay regulated.
 - Independent
 - This prevention and reliability need is unblocked by existing initiatives.
 - Negotiable
 - Definition for "regulated call volume" and "amount of cloud failures" is intentionally left open.
 - Valuable
 - Reliability when users have to escalate is crucial to consumer retention.
 - Estimable
 - Technical liaisons for the call center are available, giving the team insight into the scope of work required to satisfy operators.
 - Small
 - Similar to US-02, the technical liaison mentioned above will assist the team in managing scope.

Use Case

- UC-01 (expands US-01): PR Review Agent Consults the Knowledge Base During a Pull Request Review
 - Primary actor
 - Software Engineer (cloud team), as the pull request author
 - Secondary actors
 - PR review agent
 - GitHub-hosted engineering knowledge base
 - Preconditions
 - Team documentation has been migrated from Confluence into the GitHub knowledge base and is indexed by the PR review agent.
 - Engineer has opened a pull request against the protected branch.
 - Main Success Flow
 1. Engineer opens a pull request against the protected branch.
 2. System (PR review agent) retrieves the knowledge base articles relevant to the changed files.
 3. System posts inline review comments, each citing the specific knowledge base article backing the suggestion, within 5 minutes of PR creation.
 4. Engineer addresses the flagged comments and pushes an update.
 5. System re-evaluates the updated diff against the same knowledge base articles and marks the review approved when no violations remain.
 - Alternate Flow
 - A2: A comment is informational only. Engineer dismisses it with a one-line justification instead of pushing a new commit, and the agent marks it resolved.
 - Exception Flow
 - E1: The knowledge base index is stale or unreachable when the PR is opened. System labels the PR "knowledge base unavailable - manual review required" and notifies the author within 10 minutes.
 - Postcondition
 - The pull request has a recorded pass/fail review outcome with citations, and its review cycle time is captured for productivity metrics.

- UC-02 (expands US-01): Audit Recipe Records and Remove the Ones Below Quality Standard
 - Primary actor
 - Software Engineer (cloud team)
 - Secondary actors
 - AI review agent
 - Recipe database
 - Preconditions
 - The recipe database contains records with an instructions field and an ingredients list, queryable in batches.
 - A quality standards ruleset (e.g. minimum instruction step count, required ingredient fields) is defined and available to the review agent.
 - Main Success Flow
 1. Engineer triggers the recipe quality-review agent against the recipe database.
 2. System (agent) queries the database in batches and scores each recipe's instructions and ingredients against the quality ruleset.
 3. System flags every recipe scoring below 70 out of 100 (score subject to change), listing the specific missing or malformed field for each.
 4. Engineer reviews the batch of flagged recipes alongside the agent's justification.
 5. Engineer confirms deletion, and the agent removes the flagged recipes from the database and reports the count removed.
 - Alternate Flow
 - A2: Engineer disagrees with a flagged recipe's score. Engineer marks it as an exception to keep, and the agent excludes it from future flagging runs.
 - Exception Flow
 - E1: A batch query to the recipe database times out (exceeds 30 seconds) or fails mid-run. Agent halts, checkpoints the last successfully scored batch, and notifies the engineer of the resume point without deleting further records.
 - Postcondition
 - The recipe database contains only recipes meeting the quality standard, with a log of removed record IDs and the reason each was removed.

Acceptance Criteria

- AC-01.1 (UC-01 main flow)
 - Given a pull request is opened against the protected branch with the knowledge base indexed,
 - When the PR review agent completes its automated pass,
 - Then it posts an inline comment citing a knowledge base article for each flagged issue within 5 minutes of PR creation.
- AC-01.2 (UC-01 exception flow)
 - Given the knowledge base index is unavailable when a pull request is opened,
 - When the review agent attempts its automated pass,
 - Then it labels the PR "knowledge base unavailable - manual review required" and notifies the author.
- AC-02.1 (UC-02 main flow)
 - Given the recipe database contains records with instructions and ingredients fields,
 - When the quality-review agent scores a batch of recipes against the standards ruleset,

- Then every recipe scoring below 70 out of 100(scoring subject to change) is flagged with its specific missing or malformed field for the engineer to confirm deletion.
- AC-02.2 (UC-02 exception flow)
 - Given the agent is querying the recipe database in batches,
 - When a batch query exceeds a 30-second timeout or fails mid-run,
 - Then the agent halts the run, checkpoints the last successfully scored batch, and notifies the engineer of the resume point without deleting further records.